Bulut Bilişim ve Uygulamaları Dersi

Proje 2:

İHA Telemetri Verilerinin Gerçek Zamanlı İşlenmesi ve Görselleştirilmesi (AWS IoT Simülasyonu)

Halime Şahnur ÇÖĞÜR

23291625

github linki: https://github.com/sahnurcgr/BulutProje_2.git

video linki: https://youtu.be/6Tc59__5dnA

İÇİNDEKİLER

1. GİRİŞ VE PROJE AMACI 3

2. KAPSAM VE KULLANILAN TEKNOLOJİLER 3

3. SİSTEM MİMARİSİ VE AKIŞI 4

4. YEREL ORTAM KURULUMU VE BULUT ENTEGRASYONU 4

4.1. Veri Simülasyonu ve Bağımlılık Yönetimi 4

4.2. Güvenlik ve Kimlik Yönetimi (AWS IAM) 5

4.3. Veri Kanalı Kurulumu (AWS Kinesis Data Stream) 5

5. SUNUCUSUZ VERİ İŞLEME (AWS LAMBDA) 5

6. GERÇEK ZAMANLI VERİ DEPOLAMA (AWS DYNAMODB) 6

7. VERİ GÖRSELLEŞTİRME VE İZLEME PANELİ (DASHBOARD) 7

8. SİSTEM TESTİ VE SONUÇ 8

1. GİRİŞ VE PROJE AMACI

Bu proje kapsamında, bir İnsansız Hava Aracı (İHA) tarafından üretilen telemetri verilerinin (irtifa, hız, konum, batarya durumu vb.) gerçek zamanlı olarak bir bulut platformuna aktarılması, işlenmesi, kalıcı olarak depolanması ve son kullanıcıya bir panel üzerinden sunulması hedeflenmektedir.

Günümüzde IoT cihazları ve otonom sistemler saniyede binlerce veri noktası üretmektedir. Bu verilerin düşük gecikme süresiyle işlenmesi, operasyonel kararların anında alınabilmesi ve geçmişe dönük analizlerin yapılabilmesi için bulut tabanlı veri akış mimarileri kritik öneme sahiptir. Bu çalışma "Simüle Edilmiş Veri Üretimi -> Akış Yönetimi -> Sunucusuz İşleme -> NoSQL Depolama -> Canlı Görselleştirme" döngüsünü uygulamalı olarak göstermek amacıyla tasarlanmıştır.

2. KAPSAM VE KULLANILAN TEKNOLOJİLER

Projede uçtan uca veri akışını sağlamak için aşağıdaki modern bulut bilişim teknolojileri tercih edilmiştir:

- **Veri Kaynağı (Backend):** Python dili kullanılarak hazırlanan bir simülasyon scripti.
- **Bulut Entegrasyonu:** AWS SDK (Boto3) kütüphanesi.
- **Veri Akış Katmanı (Streaming):** AWS Kinesis Data Streams (Gerçek zamanlı veri akışı ve mesaj kuyruğu).
- **İşleme Katmanı (Compute):** AWS Lambda (Sunucusuz, olay güdümlü veri işleme).
- **Depolama Katmanı (Database):** AWS DynamoDB (Yüksek performanslı, esnek NoSQL veritabanı).
- **Kullanıcı Arayüzü (Frontend):** Python Streamlit ve Pandas kütüphaneleri.

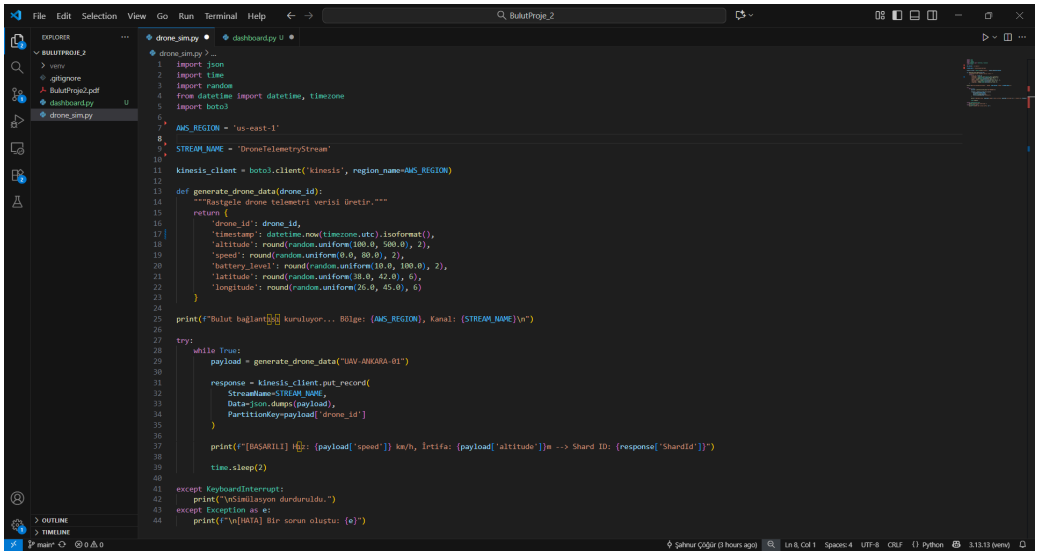
3. SİSTEM MİMARİSİ VE AKIŞI

Sistemin çalışma mantığı beş temel aşamadan oluşmaktadır:

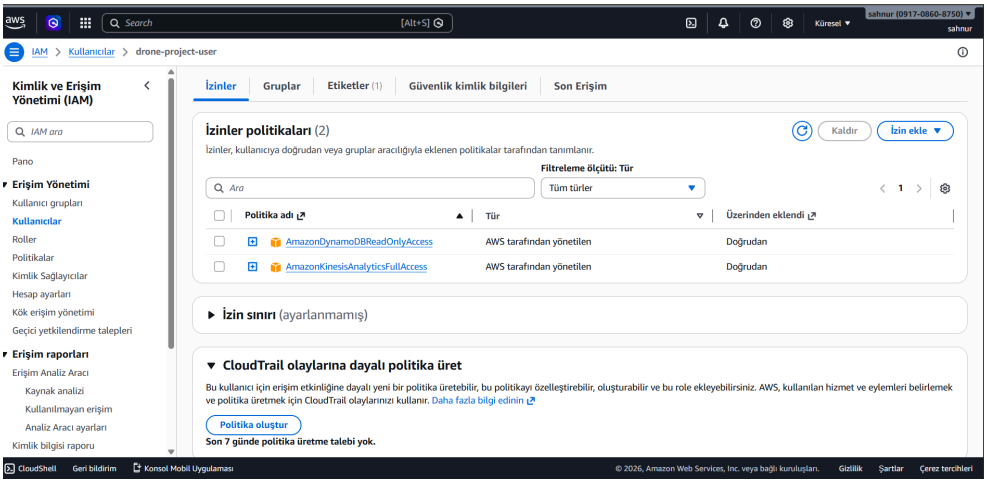
1. **Veri Üretimi:** Python scripti, rastgele koordinatlar ve sensör değerleri üreterek gökyüzündeki bir İHA'nın uçuş durumunu taklit eder.
2. **Veri Alımı:** Üretilen JSON formatındaki veriler, AWS Kinesis üzerindeki ilgili "Shard"lara (veri kanallarına) anlık olarak fırlatılır.
3. **İşleme:** Kinesis'e düşen her kayıt, bir AWS Lambda fonksiyonunu tetikler. Lambda, şifrelenmiş ham veriyi okur, veri tiplerini veritabanına uygun hale getirir (Float -> Decimal dönüşümü).
4. **Depolama:** İşlenen veri drone_id ve timestamp anahtarlarıyla DynamoDB tablosuna kalıcı olarak yazılır.
5. **Görselleştirme:** Yerelde çalışan Streamlit arayüzü, DynamoDB'yi sürekli tarayarak en güncel verileri çeker ve grafikleri/haritayı günceller.

4. YEREL ORTAM KURULUMU VE BULUT ENTEGRASYONU

4.1. Veri Simülasyonu ve Bağımlılık Yönetimi: Projenin kodlama aşamasına, Python kullanılarak yalıtılmış bir sanal ortam oluşturulmasıyla başlanmıştır. Bu sayede sistem bağımlılıkları izole edilmiş ve projeye boto3 kütüphanesi dahil edilmiştir. Yazılan Python scripti, random ve datetime kütüphanelerini kullanarak Türkiye koordinatlarına uygun enlem/boylam, 100-500m arası irtifa ve anlık hız gibi değerleri JSON formatında, saniyede bir üretecek şekilde ayarlanmıştır.



4.2. Güvenlik ve Kimlik Yönetimi (AWS IAM): Bulut ortamına veri gönderiminden önce Least Privilege prensibine uygun olarak AWS IAM üzerinden projeye özel bir kullanıcı (drone-project-user) oluşturulmuştur. Güvenlik zafiyeti oluşmaması için "Access Key" ve "Secret Key" bilgileri kod içerisine gömülmemiş yerel bilgisayardaki .aws/credentials dizininde güvenli bir şekilde saklanmıştır.

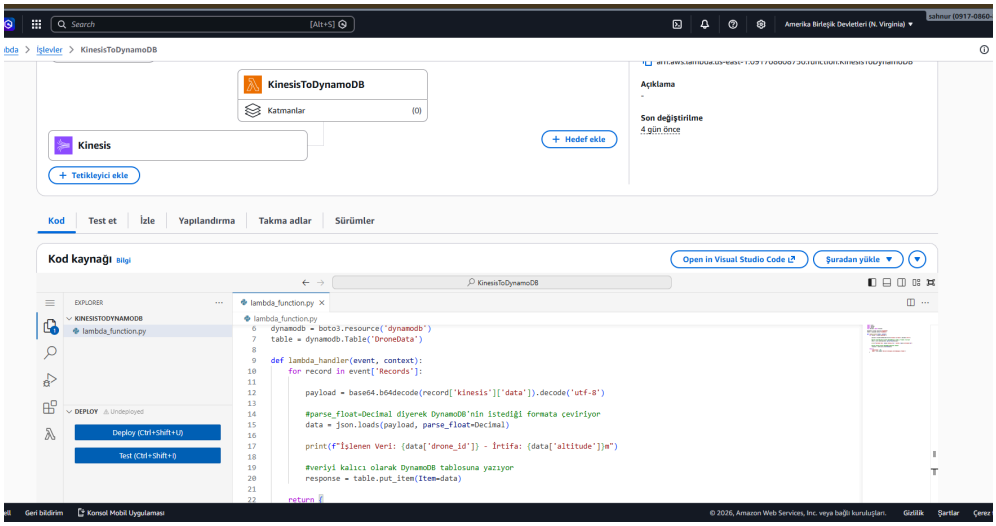


4.3. Veri Kanalı Kurulumu (AWS Kinesis Data Stream): Simülasyondan gelecek yüksek hızlı verileri karşılamak için AWS üzerinde DroneTelemetryStream adında bir veri kanalı oluşturulmuştur. Öğrenci bütçesi dikkate alınarak sistem "Provisioned" modda ve sadece 1 Shard kapasitesiyle sınırlandırılmıştır. Simülasyon çalıştırıldığında, dönen "Shard ID" yanıtlarıyla verilerin buluta ulaştığı doğrulanmıştır.

5. SUNUCUSUZ VERİ İŞLEME (AWS LAMBDA)

Kinesis veri kanalına akan ham telemetri verilerini anlık olarak işlemek ve veritabanına aktarmak için AWS Lambda kullanılmıştır.

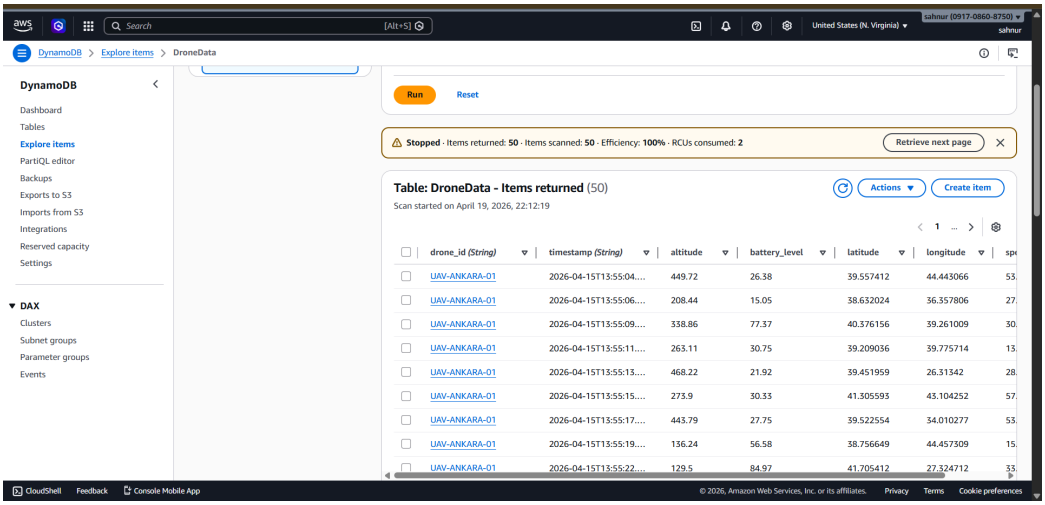
- **Tetikleyici (Trigger) Mekanizması:** Lambda fonksiyonu Kinesis kanalına yeni bir veri paketi düştüğünde otomatik olarak tetiklenecek şekilde yapılandırılmış, böylece sadece işlem yapıldığı anlarda maliyet oluşturulması sağlanmıştır.
- **Veri Dönüşümü ve Debugging:** Kinesis üzerinden güvenlik amacıyla Base64 formatında şifrelenmiş olarak gelen veriler Lambda içerisinde decode edilerek JSON formatına çevrilmiştir. AWS DynamoDB'nin standart float tiplerini desteklememesi nedeniyle geliştirme aşamasında karşılaşılan hata, Python decimal kütüphanesi entegre edilerek çözülmüştür.
- **IAM Rolleri:** Lambda'nın çalışabilmesi için "Kinesis ReadOnly" ve "DynamoDB FullAccess" yetkilerini içeren özel bir Yürütme Rolü tanımlanmıştır.



6. GERÇEK ZAMANLI VERİ DEPOLAMA (AWS DYNAMODB)

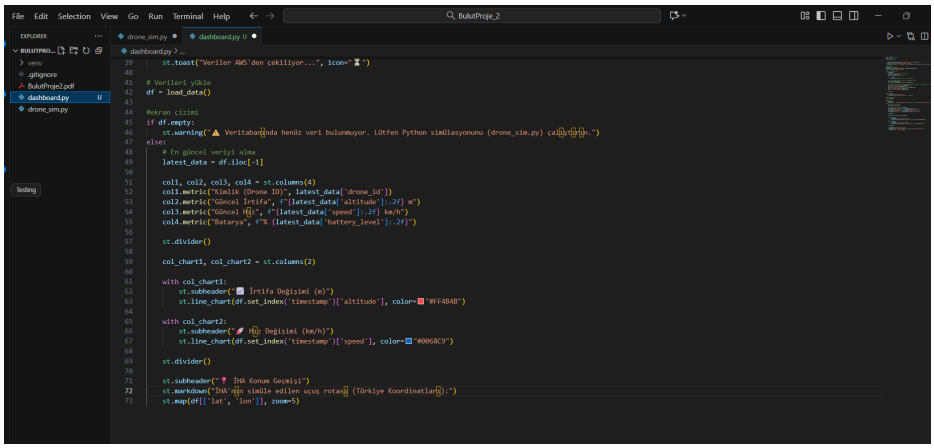
İşlenen verilerin yüksek performansla saklanması için NoSQL tabanlı AWS DynamoDB tercih edilmiştir.

- **Veri Modelleme (Data Modeling):** Zaman serisi verilerinin indekslenmesi için tablo oluşturulurken Bölüm Anahtarı (Partition Key) olarak cihaz kimliği (drone_id), Sıralama Anahtarı (Sort Key) olarak ise zaman damgası (timestamp) kullanılmıştır.
- **Kapasite Modeli:** Veritabanı "On-Demand" (İsteğe Bağlı) kapasite modunda kurularak maliyet optimizasyonu sağlanmıştır.

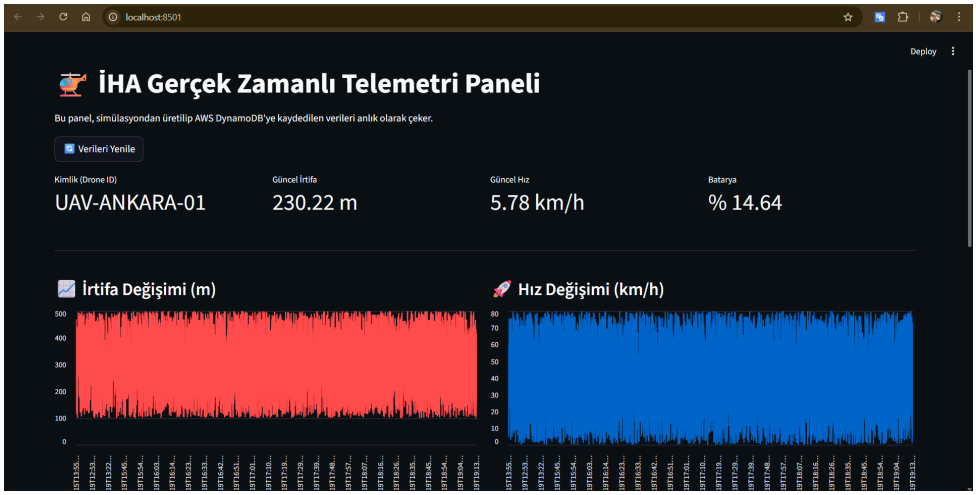


7. VERİ GÖRSELLEŞTİRME VE İZLEME PANELİ (DASHBOARD)

Projenin yalnızca backend mimarisiyle sınırlı kalmaması ve toplanan telemetri verilerinin son kullanıcıya anlamlı bir şekilde sunulması için Python Streamlit kullanılarak gerçek zamanlı bir UI (Kullanıcı Arayüzü) geliştirilmiştir.



Ekstra bulut sunucusu (EC2) maliyeti yaratmamak adına yerelde çalışan bu arayüz, DynamoDB'yi sürekli tarayarak en güncel verileri çekmektedir. Panel üzerinde İHA'nın anlık irtifa ve hız değişimleri çizgi grafiklerle (line chart) gösterilmiş, anlık koordinat verileri ise dinamik bir harita üzerinde konumlandırılarak tam teşekküllü bir IoT kontrol merkezi yaratılmıştır.



8. SİSTEM TESTİ VE SONUÇ

Tüm sistemin uçtan uca (End-to-End) testi başarıyla gerçekleştirilmiştir. Python scripti çalıştırıldığında üretilen verilerin Kinesis'e aktarıldığı, Lambda tarafından saniyenin altında bir gecikmeyle işlenip DynamoDB'ye yazıldığı ve Streamlit paneli üzerinden anlık olarak görselleştirildiği kanıtlanmıştır.

Bu proje ile gerçek zamanlı veri akışı yönetimi, AWS sunucusuz mimarilerinin entegrasyonu, NoSQL veri modelleme ve verinin değere dönüştürülüp görselleştirilmesi hedeflerine ulaşılmıştır.